FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	HERNAEZ, Angelee Louise R.
Section:	AN43
Professor:	Mrs. Crisola Tan
Date:	October 12, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

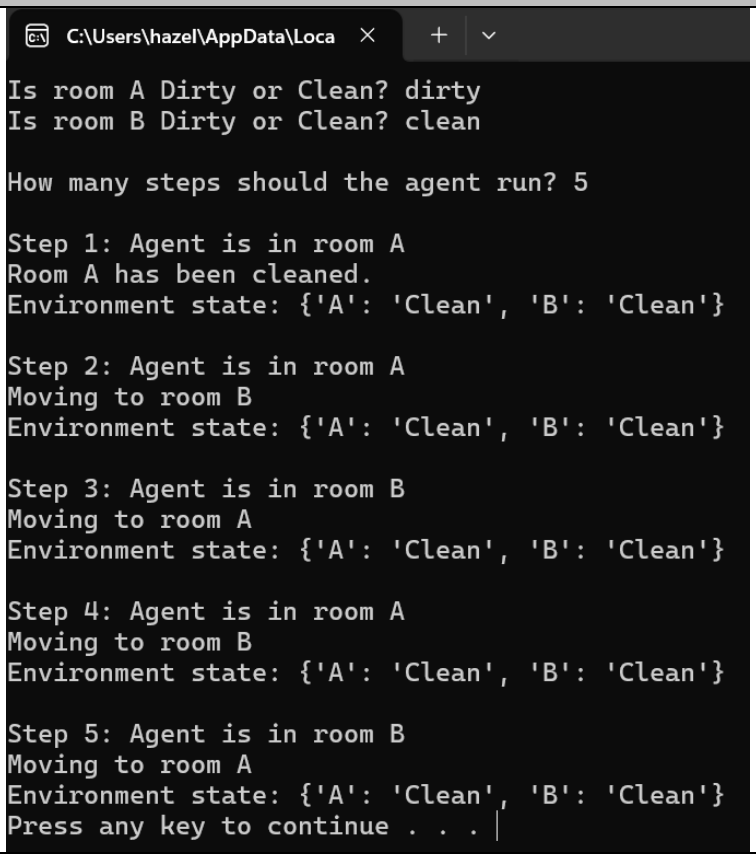
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

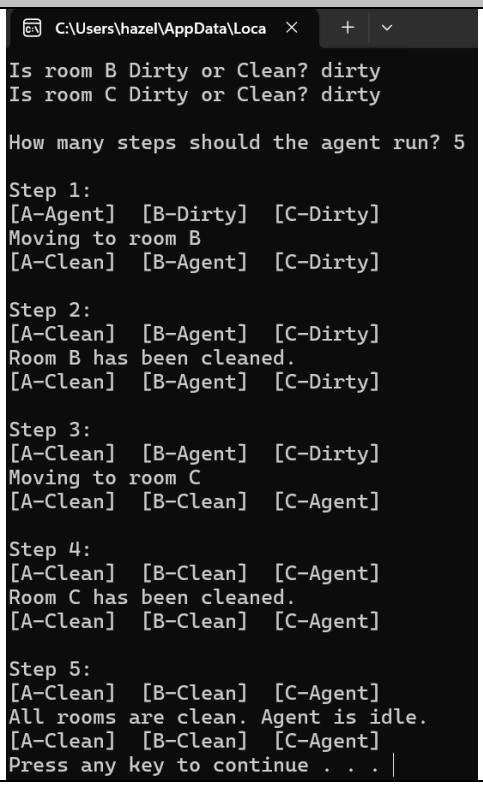
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

Source Code:

```
#Task 1: Set Up Environment

#Initialize two rooms
rooms = {
    "A": "",
    "B": ""
}

#Let the user set the initial state of each room
for room in rooms:
    state = input(f"Is Room {room} dirty or clean? ").lower()

    while state not in ["dirty", "clean"]:
        state = input("Invalid input. Enter dirty or clean: ").lower()

    rooms[room] = state

#function: check if a room is dirty
def is_dirty(room):
    return rooms[room] == "dirty"

#function: clean a room
def clean_room(room):
    rooms[room] = "clean"


#Task 2: Implement Rule-Based Agent
class Agent:
    def __init__(self):
        self.location = "A"

    #function: move to the other room
    def move(self):
        if self.location == "A":
            self.location = "B"
        else:
            self.location = "A"

    #function: perceive and act based on the room's state
```

```
    def perceive_and_act(self):
        if is_dirty(self.location):
            clean_room(self.location)
            action = "Cleaned Room " + self.location
        else:
            self.move()
            action = "Moved to Room " + self.location

        return action


#Task 3: Run Simulation

agent = Agent()

steps = int(input("\nHow many steps should the simulation run? "))

for step in range(steps):
    print(f"\nStep {step + 1}")

    action = agent.perceive_and_act()

    print("Agent Location:", agent.location)
    print("Action Taken:", action)
    print("Environment State:", rooms)
```

Output:

```
Is Room A dirty or clean? dirty
Is Room B dirty or clean? dirty

How many steps should the simulation run? 6

Step 1
Agent Location: A
Action Taken: Cleaned Room A
Environment State: {'A': 'clean', 'B': 'dirty'}

Step 2
Agent Location: B
Action Taken: Moved to Room B
Environment State: {'A': 'clean', 'B': 'dirty'}

Step 3
Agent Location: B
Action Taken: Cleaned Room B
Environment State: {'A': 'clean', 'B': 'clean'}

Step 4
Agent Location: A
Action Taken: Moved to Room A
Environment State: {'A': 'clean', 'B': 'clean'}

Step 5
Agent Location: B
Action Taken: Moved to Room B

Step 6
Agent Location: A
Action Taken: Moved to Room A
Environment State: {'A': 'clean', 'B': 'clean'}
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. The agent follows two rules; if the room is dirty, it cleans the room. And if the room is clean, it moves to the other room.

2. The agent checks the state of its current room. If it is dirty, the agent cleans it; otherwise, it moves to the other room.

3. If both the rooms are already clean from the start, the agent will keep moving between rooms until it runs out of moves, because it does not have to clean anything.

4. I would modify the agent to use a list or dictionary containing multiple rooms. I would also update the movement method so the agent can move between all available rooms instead of switching only between Rooms A and B.